

Proyecto 9**Sistema de gestión de restaurante
(versión para el 1er parcial, solo consola)****Objetivo y alcance**

Desarrollar en Java (solo consola) un sistema que gestione el funcionamiento básico de un restaurante, permitiendo administrar menús, pedidos, mesas, clientes y meseros. El propósito es aplicar los principios de la Programación Orientada a Objetos (POO): encapsulamiento, herencia, polimorfismo, uso de interfaces, colecciones y validaciones de reglas de negocio.

Requisitos funcionales mínimos

1. Registrar, listar y buscar platos del menú.
2. Registrar clientes y meseros.
3. Asignar mesas a clientes, controlando capacidad y disponibilidad.
4. Crear pedidos asociados a una mesa y un cliente.
5. Agregar o eliminar platos de un pedido (interfaz Ordenable).
6. Calcular el total de la cuenta con posibles descuentos (interfaz Pagable).
7. Generar reportes de platos más vendidos y mesas más ocupadas.
8. Mostrar los pedidos activos y su estado (en preparación, servido, pagado).

Requisitos no funcionales

- Versión mínima: Java 17+.
- Ejecución: solo en consola, sin frameworks ni GUI.
- Código: en español, con uso correcto de toString(), equals() y hashCode().
- Validaciones: mediante excepciones personalizadas y manejo de errores controlado con try/catch en main.

Reglas de negocio

1. Cada mesa tiene una capacidad máxima de personas y un estado (LIBRE, OCUPADA, RESERVADA).
2. No se puede asignar una mesa ocupada a otro cliente.
3. Cada pedido debe incluir al menos un plato del menú.
4. Los platos pueden pertenecer a diferentes categorías: Entrada, Plato Fuerte, Postre o Bebida.
5. Los precios de los platos son unitarios y se deben sumar al calcular la cuenta.
6. Se puede aplicar un descuento si el cliente cumple alguna condición (por ejemplo, más de cierto monto o cliente frecuente).
7. Cada pedido está vinculado a una mesa y a un cliente.
8. Los reportes deben incluir:
 - Platos más vendidos.
 - Mesas más utilizadas.
 - Ventas totales por día.

Aspectos clave de implementación

- Herencia: Persona y Plato como clases abstractas para promover reutilización y jerarquía.
- Polimorfismo: distintas subclases de Plato pueden comportarse de manera distinta (por ejemplo, en descuentos).
- Encapsulamiento: todos los atributos deben ser privados, con getters y setters.
- Validaciones:
 - No duplicar mesas ni platos.
 - No permitir mesas con capacidad excedida.
 - No crear pedidos vacíos.

Validaciones y excepciones personalizadas

- Intento de asignar mesa ya ocupada.
 - mesa no registrada.
 - número de clientes mayor al permitido.
 - plato no encontrado o fuera de menú.
 - intento de confirmar pedido sin platos.
 - cliente no encontrado.
- Todas deben manejarse en el main con mensajes claros y controlados.

Menú de consola sugerido

1. Registrar cliente
2. Registrar mesero
3. Registrar plato en el menú
4. Listar menú disponible
5. Registrar mesa
6. Asignar mesa a cliente
7. Crear pedido y agregar platos
8. Mostrar pedidos activos
9. Calcular cuenta total y aplicar descuento
10. Liberar mesa y marcar pedido como pagado
11. Reporte: platos más vendidos
12. Reporte: mesas más ocupadas
13. Salir

Datos de prueba mínimos

- Platos:
 - Entrada: Ensalada César (\$4.00)
 - PlatoFuerte: Lomo Fino (\$12.00)
 - Postre: Tarta de Queso (\$5.00)
 - Bebida: Jugo Natural (\$2.00)
- Mesas:
 - Mesa 1: capacidad 4
 - Mesa 2: capacidad 6
- Clientes:
 - María López (frecuente)
 - Juan Pérez (eventual)
- Meseros:
 - Carlos Ruiz (\$500, 2 mesas)
- Casos de prueba:
 - Pedido válido con varios platos.
 - Intento de exceder capacidad de mesa (debe lanzar excepción).
 - Pedido vacío (debe lanzar excepción).
 - Cálculo de cuenta con descuento.

Reportes requeridos

- Platos más vendidos (ranking).
- Mesas más ocupadas.
- Total de ventas del día.
- Promedio de consumo por mesa.
- Historial de pedidos por cliente.

Extras opcionales (para subir nota)

- Persistencia simple en archivos .txt o .csv.

Lo que no se debe desarrollar en este parcial

- Interfaz gráfica (Swing o JavaFX).
- Conexión a base de datos.
- Hilos o procesamiento concurrente.
- Facturación real o pago electrónico.
- Patrones avanzados (Observer, MVC, Factory, etc.).

Plan de implementación por etapas

Semana 1

- Modelo base: Persona, Cliente, Mesero, Plato (abstracta) y sus subclasses.
- Clases Mesa y Pedido.
- Encapsulamiento, toString(), equals(), hashCode().

Semana 2

- Clase Restaurante: gestión de menús, mesas, pedidos, clientes y meseros.
- Implementación de interfaces Ordenable y Pagable.
- Validaciones y excepciones personalizadas.
- Menú de consola básico.

Semana 3

- Implementar reportes (platos más vendidos, mesas más usadas, total de ventas).
- Casos de prueba controlados.
- Limpieza de código y documentación final.

Criterios de evaluación

- Correctitud funcional y validaciones: 35%
- Diseño POO (herencia, polimorfismo, interfaces, encapsulamiento): 30%
- Calidad del código (toString, equals, manejo de excepciones): 20%
- Reportes y pruebas en consola (main): 15%

Comentarios de diseño

- Plato debe ser abstracta para representar un menú extensible y polimórfico.